



Chapter 3

Inter-Trip Communication and Synchronization

Cooperative itineraries

- Processes that share information with other processes or affect commands executed by other processes
- **Inter-trip communication** : There must be some communication mechanism between collaborative processes to pass the required instructions or data
- **Synchronization** : Coordination issues and handling mechanisms involved in the process of itinerary cooperation

3-1 Inter-trip communication (IPC)

- **Shared files** : The most common way to share information between processes is that the amount of information passed is limited only by the maximum length of the file system
- **Shared memory** : The operating system provides a mechanism that allows different processes to access a certain piece of memory at the same time
 - Faster than shared files, but with limited length
 - Threads inherently contain the concept of shared memory
- **signal** : When process A wants to inform process B that something is happening or completed, it can send a specific signal to process B
 - It can be used to notify events, but if specific information needs to be sent, it needs to be used with other inter-process communication mechanisms

Messaging system - direct communication

- Both parties must know each other's identity ,
- A communication link has only 2 trips involved, which can be one-way/two-way
- The system usually provides 2 functions

`send_process(pid, message)`

`receive_process(pid, message)`

Messaging system - indirect communication

- A communication link can be shared by several processes
- There are many issues with permission management
- The system usually provides 4 functions
 - `create_mailbox(mbx)`
 - `delete_mailbox(mbx)`
 - `send_mailbox(mbx, message)`
 - `receive_mailbox(mbx, message)`

Blocked transmission vs. non-blocking reception

- Whether it is direct or indirect communication, the recipient and the recipient will face a situation where the two sides are out of sync ◦
- **Blocked Send/Blocked Receipt:** Puts the waiting party into a suspended state until the message is received before continuing
- Non-barrier communication :
 - **Non-blocking delivery** : After the sender process sends a message, it does not have to wait for the message to be received, and can continue to perform the following actions
 - **Non-blocking reception** : After the receiver process attempts to read the message, it can continue to perform subsequent actions regardless of whether the message is received or not, When both the sender and receiver use blocked communication, the two are synchronized

IPC for Windows

- **signal** : A low-level communication method between the itinerary and the outside world
- **Shared memory** : Large amounts of data communication between processes can be used, and synchronization mechanisms are required to ensure data integrity
- **Pipe** : Use communication links in the form of byte streams between processes
- **mail slot** : A non-blocking communication mechanism similar to a mailbox, using variable-length messages, is a one-way link in a master-slave architecture
- **socket** : Network communication mechanism based on TCP/IP protocol

Design considerations for inter-trip communication

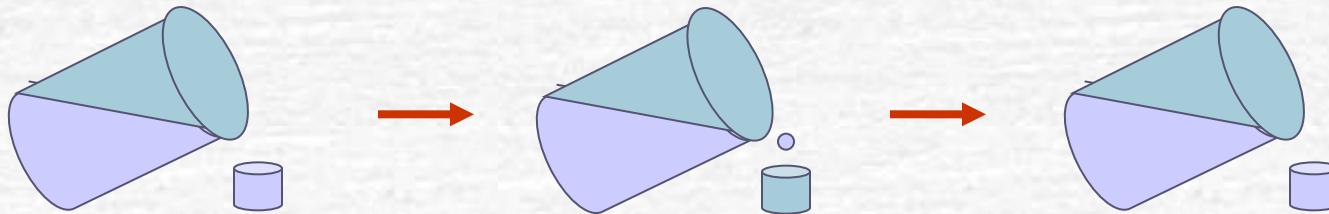
- The type of communication link : Directly or indirectly
- The number of processes that share the communication link
- Directionality of communication links : Unidirectional or bidirectional
- Message length : Fixed length or variable length
- The message is sent with a copy of the message or the reference address of the message
- What to do when a unilateral trip is terminated



Discussion Questions

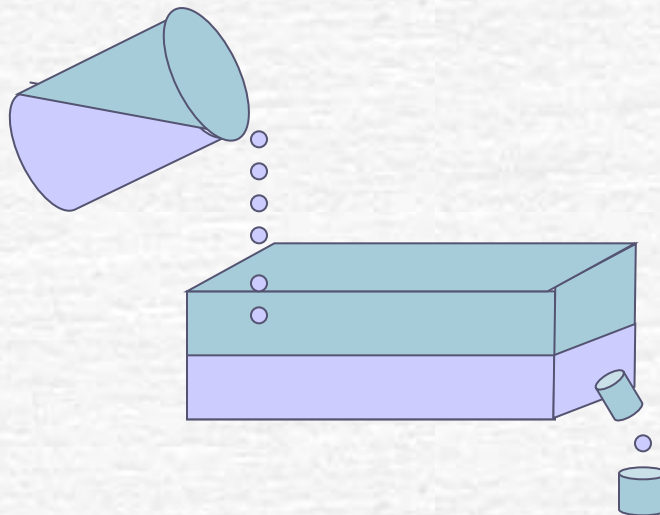
- What happens when the sender is much faster than the receiver?
- How can it be improved?

Figure 3-1 Buffering of communication links



(a) Bufferless data delivery

When the sender is much faster than the receiver, the sender is often in a waiting situation → **Buffers can be used to improve**



(b) Buffered data delivery

You must additionally verify that the data has been read

3-2 Synchronization

- Collaboration trips may interact at different points in time
- in a single-processor system : Cooperative processes are staggered execution in the CPU based on the results of the schedule, making it appear as if the results are running simultaneously
 - in multiprocessor systems : Cooperative schedules may not only be staggered, but may even be executed simultaneously
- **Synchronization mechanism** : Ensure that in an environment where multiple cooperative processes are running simultaneously, the computations of different processes do not interfere with each other

Possible problems between cooperative itineraries

- Hunger
- Dead end
- Competition status
- Inconsistencies in shared data

Critical zone

- Some sections of code run by a cooperative process involve shared resources that cannot be accessed by other processes at the same time, hence the term critical area
- The structure of the critical zone :
 - `Enter_section();` Entrance area
 - `<critical section>` Critical zone
 - `Exit_section();` Export zones
- **Entrance area** : Actions are usually taken to obtain permission to enter the critical zone
- Waiting to enter the itinerary can be busy waiting, or it can go into a suspended state, waiting to be woken up
- **Export zones** : Releases the permissions that were originally obtained so that other processes can enter the critical zone

Design conditions for critical zones

- **Mutual exclusion** : If a process is in a critical zone and that segment accesses a set of shared resources $R_1, R_2, \dots, R_n$, no other processes can enter the critical zone that accesses any of the shared resources $R_1, R_2, \dots, R_n$
- **Proceed** : If no trips are currently in the critical zone, and there are trips waiting to enter the critical zone, one of the waiting trips must be allowed to enter the critical zone for a limited time
- **Limited waiting** : The time between the request to enter the critical zone and the granted entry must be limited and cannot be prevented from entering the critical zone due to hunger or dead knots

3-3 Implementation of critical zones - method 1

- Title : Design a synchronization mechanism for two strokes P0 and P1 with the same critical area
- Practice :
 - Set the shared variable turn; When turn is 0, the stroke P0 can enter the critical area, and when the variable is 1, it is the turn of the stroke P1
 - The dummy code for P0 is as follows :

```
while (turn != 0); /* entrance area; Wait for your turn */  
<critical section>  
turn = 1; /* export zones; Set the right to use it to someone else*/
```
 - P1 can swap 0 and 1
 - Problem: Only the mutually exclusive requirements are met, but the conditions for implementation cannot be met

Implementation of critical zones - method 2

- Modify to record the current critical zone request for each journey
- P_0 virtual code :
 - `flag[0] = true;` `/* Entrance area */`
 - `while (flag[1]);`
 - `<critical section>`
 - `flag[0] = false;` `/* Export zones */`
- Problem: If one of the processes switches during the execution of the entry zone, both parties can go into an infinite loop

Figure 3-2 Implementation problems of critical zones

P_0 `flag[0] = true;`

Trip switching

P_1 `flag[1] = true;`
 `while (flag[0]);` ← Enter the loop

Trip switching

P_0 `while (flag[1]);` ← Enter the loop

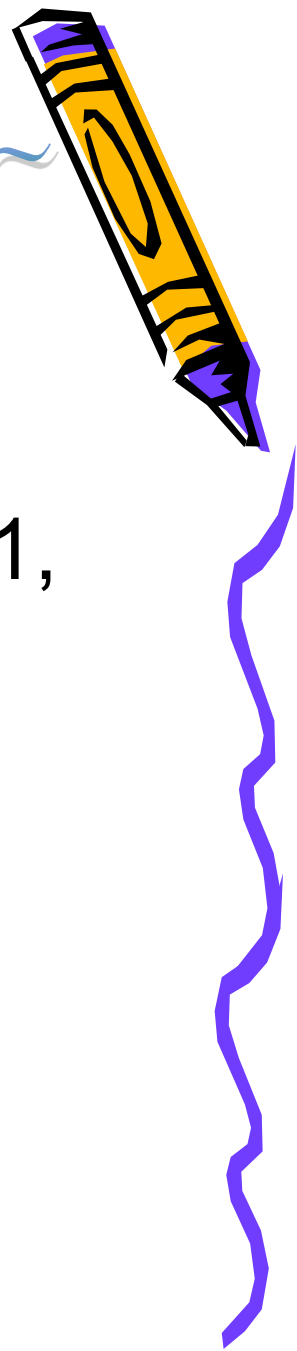
Implementation of the critical zone - bakery algorithm

- A number is issued to the process requesting the critical zone, and when the current trip using the critical zone leaves, each waiting process must first compare the numbers obtained by all waiting trips
 - If your number is the earliest, you can enter the critical zone
 - If there is a number earlier than yours, you must enter the loop and wait first
- Basically **FCFS** rules are adopted, so it can ensure that requirements with limited waits are carried out, and can be applied to critical zone implementations for multiple processes

Virtual code for bakery algorithms

- The virtual code of the itinerary P_i

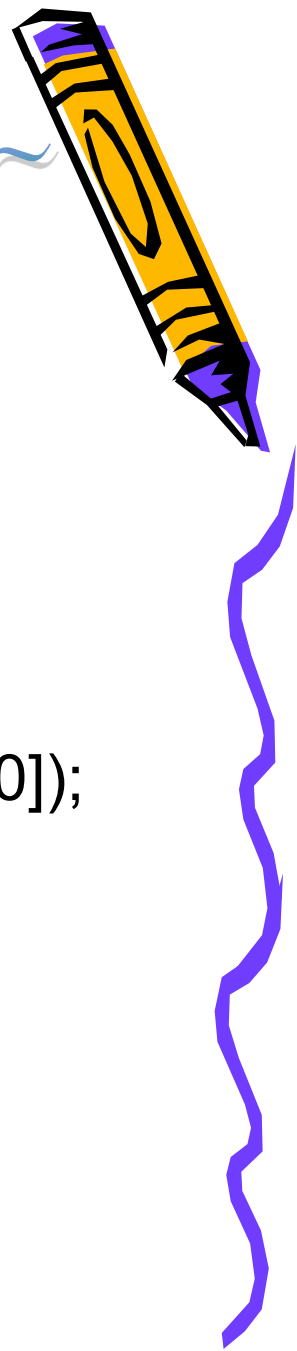
```
flag[i] = true;
number[i] = max(number[0], ..., number[n-1]) + 1; /* 取得號碼牌 */
flag[i] = false;
for (j=0; j<n; j++) /* Check the numbers of each trip one by one*/
{
    while (flag[j]); /* It means that the PD is drawing a number
ticket, and you should wait for it to finish */
    while (number[j] !=0 && ((number[j] < number[i]) ||
(number[j]=number[i] && j<i))); /* Compare numbers with
waiting*/
}
<critical section>
number[i] = 0; /* export zones; Return the numbered ticket*/
```



Classroom exercises

- Apply the bakery algorithm to the scenario of two processes P0 and P1, and write the virtual code of P0





Practice Answers

```
flag[0] = true;  
number[0] = max(number[0], number[1]) + 1;  
flag[0] = false;  
while (flag[1]);  
while (number[1] != 0 && number[1] < number[0]);  
<critical section>  
number[0] = 0;
```



Figure 3-3 Implementation example of the bakery algorithm

P_0

```
flag[0] = true;  
number[0] = 1;  
flag[0] = false;
```

← Get a numbered ticket

Trip switching

P_1

```
flag[1] = true;  
number[1] = 2  
flag[1] = false;  
while (flag[0]);  
while (number[0] != 0 && number[0] < number[1]);
```

← Get a numbered ticket

← Leave the loop

← and other numbers
are executed first

Trip switching

P_0

```
while (flag[1]);  
while (number[1] != 0 && number[1] < number[0]);  
<Critical section>
```

← Leave the loop

← Leave the loop

← Enter the critical zone

Hardware-supported synchronization mechanism

- Utilizing software entirely to implement the synchronization mechanism is not only time-consuming, but also difficult to verify its correctness
- Hardware-supported synchronization mechanism :
 - Suppress interruptions
 - TestAndSet
 - swap
 - signal

Suppress interruptions

- In a single-processor system, once the process gains control of the CPU, if it can suppress the interruption of the operating system first, it can continue to run to avoid being interrupted in the middle of the critical area
- Its dummy code :

```
Disable_Interrupt(); /* entrance area; System prima for  
suppressing interruptions ( primitive ) */
```

```
< critical section > /* Critical zone */
```

```
Enable_Interrupt(); /* export zones; Open the interrupted  
system primary*/
```

Disadvantages of the suppression interruption method

- Not suitable for multi-processor environments
- Affect the handling of the interruption
- Affect the schedule
- It can cause other hunger problems waiting to enter the critical zone itinerary

TestAndSet command

- Provide a special hardware command TestAndSet and ensure that the command will not be interrupted during execution-**Indivisibility**
- TestAndSet command :

```
boolean TestAndSet (Boolean locked) {  
    if (locked)  
        return true;  
    else {  
        locked = true;  
        return false;  
    }  
}
```

Implementation of critical zones - TestAndSet



- Use the indivisible TestAndSet command to design the critical area :

```
while (TestAndSet(lock)); /* Entrance area*/
```

```
<critical section>
```

```
lock = false; /* Export zones*/
```

- Mutual exclusion and progress are guaranteed, but limited wait is not guaranteed



TestAndSet-Improved version

`waiting[pid] = true;`

`wait_lock = true;`

`while (waiting[pid] && wait_lock)`

`wait_lock = TestAndSet(lock));` → Record trips that are waiting to enter the critical zone
Door locks in critical areas

<critical section>

`next_pid = (pid+1) % MAX_PID;`

`while (next_pid != pid && waiting[next_pid] == false)`

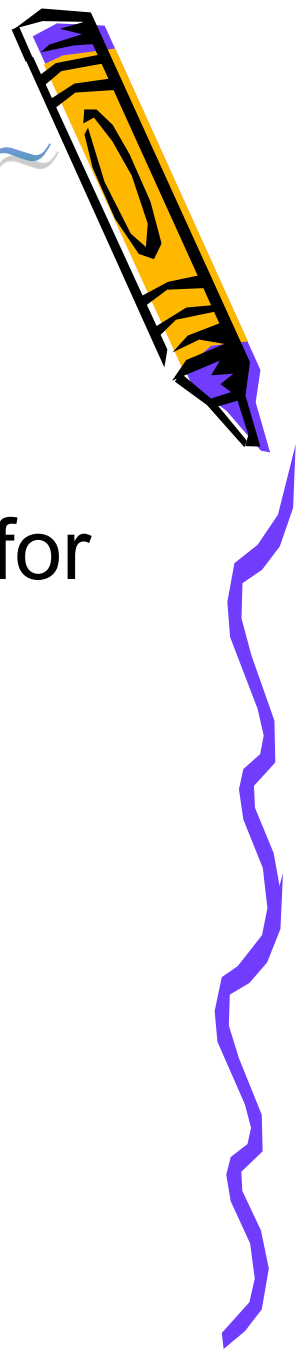
`next_pid = (next_pid+1) % MAX_PID;`

`if (next_pid == pid)`

`lock = false;`

`else`

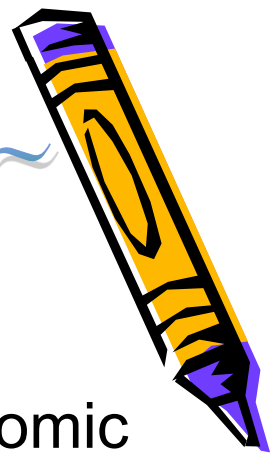
`waiting[next_pid] = false;`



Classroom exercises

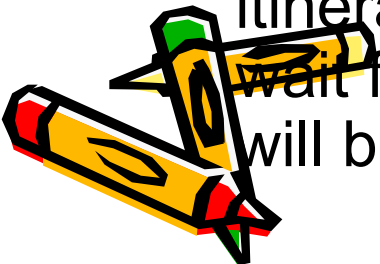
- Verify that the improved version of TestAndSet meets the three criteria for the critical area





Practice Answers

- **Mutual exclusion** : Because TestAndSet is an atomic operation, as long as one process obtains a lock, the result of other processes executing TestAndSet will be true, and it will stay in the while loop
- **Proceed** : When a trip in the critical zone comes out, if there is a process waiting, it is set according to the order of the trip code
- **waiting[next_pid] = false** , Select the next waiting trip after it
- **Limited waiting** : Since it is in the order of the itinerary code, each waiting schedule only needs to wait for N-1 schedule to be executed at most, and it will be your turn



swap

- Swap : Hardware commands that operate atomically and can exchange the contents of two words
- Swap command :

```
void swap(boolean a, boolean b)
{
    boolean temp;
    temp = b;
    b = a;
    a = temp;
}
```

Implementation of critical zones - swap

- Set the initial value of the shared variable lock to false

```
key = true;  
while (key)  
    swap (lock, key);  
<critical section>  
lock = false;
```

- Similarly, there is **no guarantee** of limited waiting

signal

- Use queues to implement blocked waits to avoid busy waits
- A signal is an integer value and has two corresponding atomic operations : **wait** With **signal**
- The value of the signal :
 - Greater than 0: Indicates that the trip can enter the critical zone
 - equal to 0: means that the capacity of the critical area is full and can no longer be entered, but no one is waiting at present;
 - Less than 0: Indicates that there are already trips waiting in the queue

```
struct semaphore {  
    int count;  
    ProcessQueue queue;  
}
```

Signal operation

- Wait :
 - void wait(semaphore s) {
 - s.count--;
 - if (s.count < 0) {
 - insert process to s.queue;
 - block(); } }
- Signal :
 - void signal(semaphore s) {
 - s.count++;
 - if (s.count <= 0) {
 - remove process P from s.queue;
 - wakeup(P); } }

Implementation of the critical zone – signal

- Set the initial value of the signal mutex to 1

```
wait(mutex);  
<critical section>  
signal(mutex);
```

- If the waiting queue adopts a first-in, first-out approach, it can ensure that the conditions for limited waiting are met; Otherwise, there is a risk of hunger
- Suppose the initial value of mutex is set to n, which means that up to n processes can be allowed to enter the critical zone at the same time
- Note: The signal provides a simple critical area synchronization mechanism, but it does not guarantee that programs will necessarily exclude each other, nor can it completely avoid deadlocks

Count signal and binary signal

- **Count signals** : signals that can be set to any positive value, also known as **Universal signal**
- **Binary signal** : It is limited to only 0 and 1 signals
- Binary signals can be implemented using hardware operations on binary values
- Binary signals can also be used to implement count signals

Utilize binary signals to implement count signals

- The integer **C** represents the value of the count signal
- Binary signal **S1** provides access control **for C** (initial value is 1)
- Binary signal **S2** simulates the waiting effect of the count signal (initial value is **0**)

```
void wait (S) {  
    wait (S1);  
    C--;  
    if (C < 0) {  
        signal (S1);  
        wait (S2);  
    }  
    signal (S1);  
}
```

```
void signal (S) {  
    wait (S1);  
    C++;  
    if (C <= 0)  
        signal (S2);  
    else  
        signal (S1);  
}
```

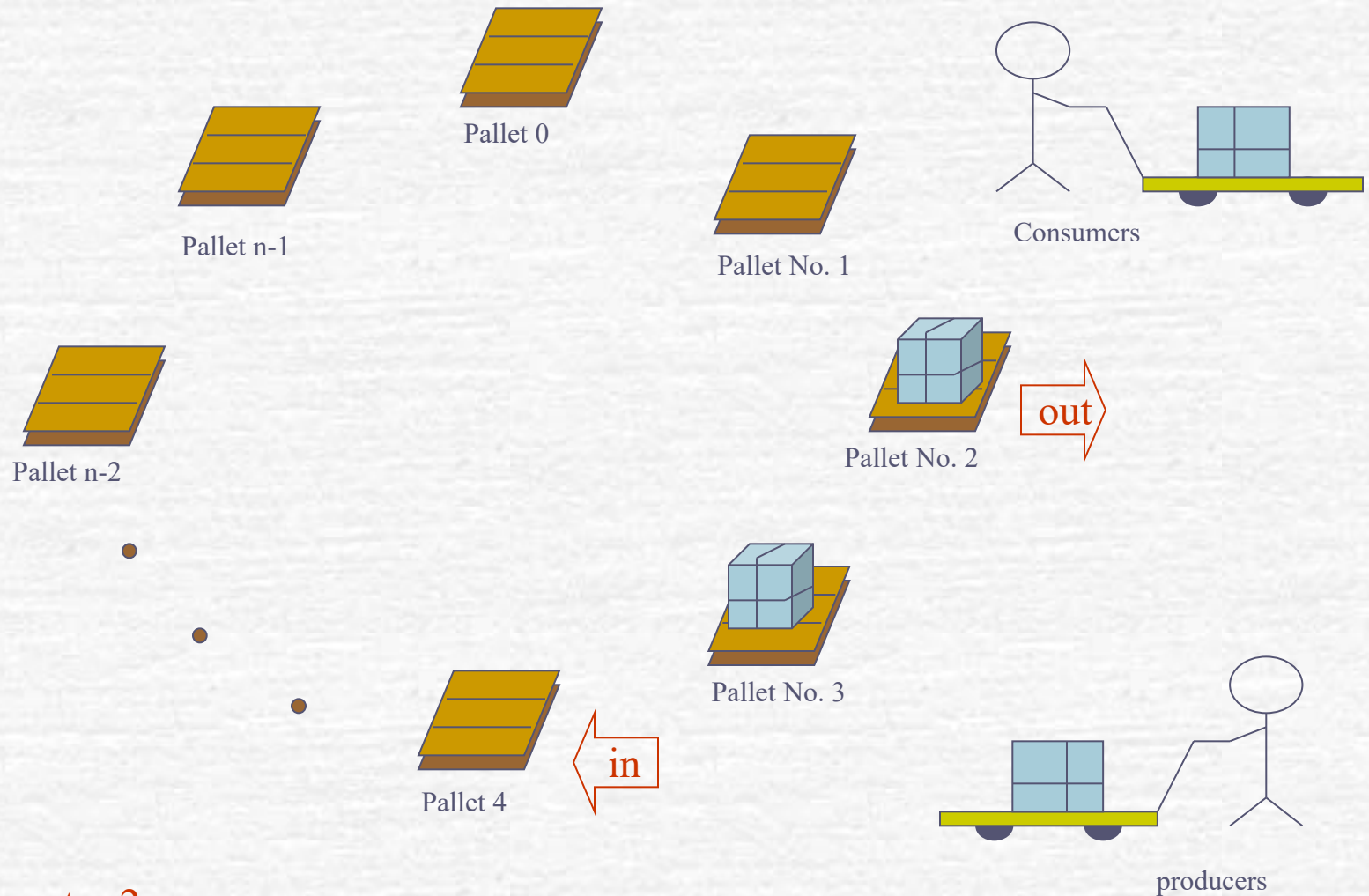
3-4 Classic examples of synchronization problems

- Limited buffer problem
- Reader vs. writer questions
- The philosopher's dinner question

Limited buffer problem

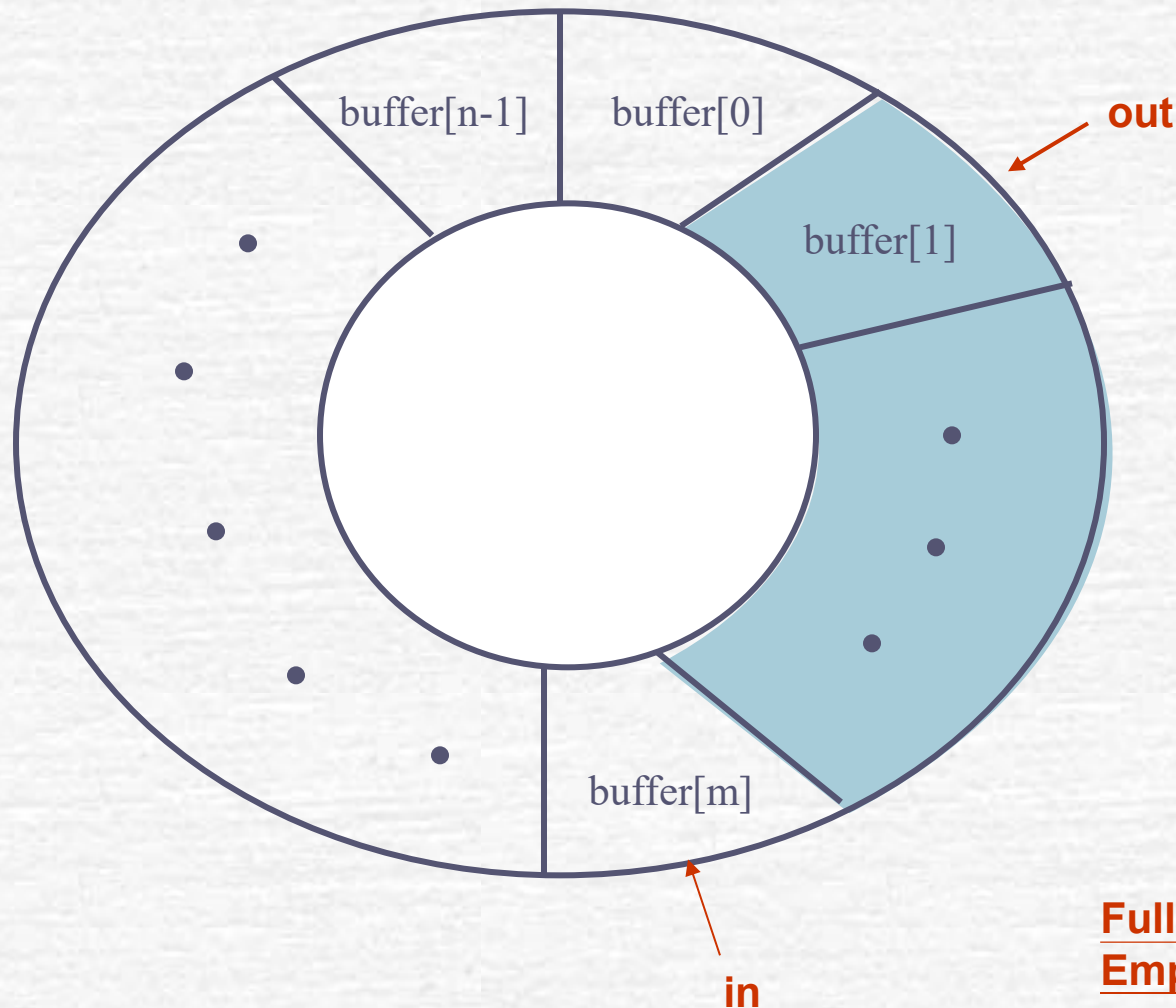
- Assuming that up to n items of data can be stored in a buffer with limited space, it is like having a total of n pallets in a warehouse, which can be used to store n goods, as shown in **Figure 3-4**
- In this warehouse, there is a producer process that puts the data into the pallet, and another consumer process that takes the data out
- Objectives :
 - Producers can pause production when the warehouse is full
 - Consumers can wait when the warehouse is empty
 - Both producers and consumers can access data correctly without errors

Figure 3-4 Example of a finite buffer



count = 2

Figure 3-5 Ring Queue Implementation for Finite Buffers



Full = m - 1
Empty = n - m + 1

Finite buffer implementation – Producer (Critical zone control)

```
while (1)
{
    Production of submission data
    next_pdata;
    while (counter == n);
    buffer[in] = next_pdata;
    in = (in + 1) % n;
    counter++;
}
```

Limited buffer implementation – consumer (Critical zone control)

```
while (1)
{
    while (counter == 0);
    next_cdata = buffer[out];
    out = (out + 1) % n;
    counter--;
    Use the removed
    next_cdata;
}
```

Issues with limited buffer implementation

- Suppose a producer is generating a new piece of data, and a consumer is reading a piece of data
- If a process switch occurs while both of these are updating the **counter**, the value of the **counter** may be incorrect
- Critical zones can be used to solve this problem
 - Universal signal **empty** : Represents the total number of blank fields, with an initial value of n
 - Universal signal **full** : Represents the total number of fields that have been populated, with an initial value of 0
 - Binary signal **mutex** : As an access control to the buffer

The rewritten producer program

```
while (1)
{
    Production of submission data
    next_pdata;
    wait (empty);
    wait(mutex);
    buffer[in] = next_pdata;
    in = (in + 1) % n;
    signal(mutex);
    signal(full);
}
```

Rewritten consumer program

```
while (1)
{
    wait(full);
    wait(mutex);
    next_cdata = buffer[out];
    out = (out + 1) % n;
    signal(mutex);
    signal(empty);
    Use the removed
    next_cdata;
}
```

Reader vs. writer questions

- Suppose there are several processes sharing the same data object, some of which only need to read the data, called the reader, and some of which are only responsible for updating the data, called the writer
- If two readers read the data at the same time, it doesn't cause any problems
- However, if a reader is reading data while a writer is writing data, or another writer is writing data, it can cause data inconsistency
- Objectives :
 - When one writer changes data, no other processes can read or write the data
 - When any number of readers read the data, no writer can write
 - There are no restrictions between readers and readers

Breaking area implementation of the reader vs. writer problem

- Shared variable `read_cnt`: The number of readers who are currently reading the shared data in the record, with an initial value of 0
- Signal mutex: Used to control access to `read_count`, with an initial value of 1
- Signal `wrt`: Used to control the synchronization when writing data, the initial value is 1
- The writer's code :

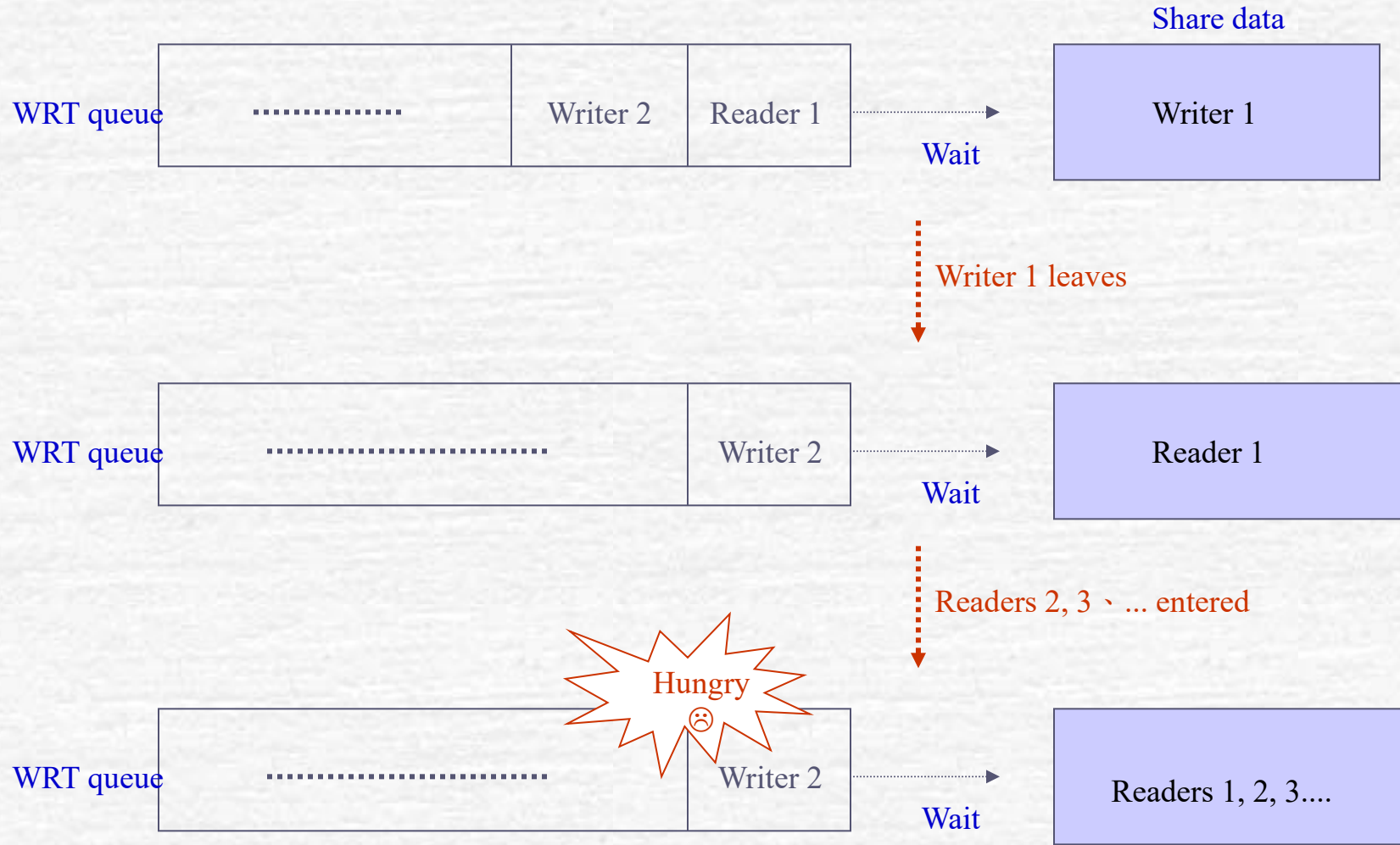
```
wait(wrt);  
Write data  
signal(wrt);
```

The reader's code

```
wait(mutex);  
read_cnt++;  
if (read_cnt == 1)  
    wait(wrt);  
signal(mutex);  
Read the data  
wait(mutex);  
read_cnt--;  
if (read_cnt == 0)  
    signal(wrt);  
signal(mutex);
```

- It can solve the problem of **data inconsistency**, but it may cause **hunger**

Figure 3-7 Example of a starving writer



Error-prone signal programming

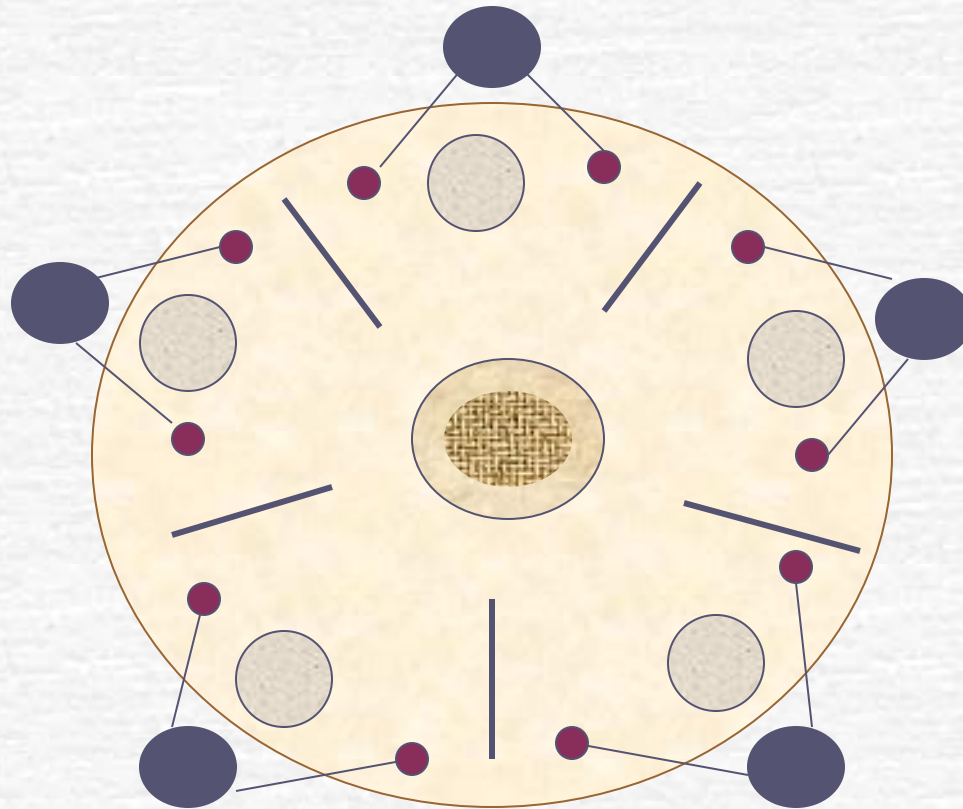
- Taking the reader and writer as an example, if the order of the release of WRT and the release of mutex is exchanged in the program, an error will occur :

```
wait(mutex);  
read_cnt--;  
signal(mutex);  
if (read_cnt == 0)  
    signal(wrt);
```

The philosopher's dinner question

- Suppose there are 5 philosophers sitting around a round table for dinner, and there are only 5 chopsticks on the table, and every 2 philosophers share 1 chopstick
- Philosophers sit at the table and still ponder philosophical questions
- When the philosopher felt hungry, he would pick up a chopstick from the left and right to eat
- Philosophers with 2 chopsticks can eat at the same time, but they cannot snatch chopsticks from other people's hands
- After eating, the philosopher will put down both chopsticks and continue to think about philosophical questions, while other philosophers can use the chopsticks that have been put down

Figure 3-9 Philosopher's Dinner



Implement the customer's dinner with the signal

- Each chopstick is represented by a signal (chopstick[i]), and their initial value is set to 1
- The code of the ith customer :

```
do {  
    wait(chopstick[i]);  
    wait(chopstick[ (i + 1) % 5]);  
    Eat;  
    signal(chopstick[i]);  
    signal(chopstick[ (i + 1) % 5]);  
    Think;  
} while (1);
```

The philosopher's deadLocks

- A dead knot occurs when each customer picks up the chopsticks in their right hand and waits for the chopsticks in their left hand
- Possible workarounds :
 - Place $n+1$ chopsticks on the table, or limit a maximum of $n-1$ customers to eat at the same time ◦
 - It is stipulated that customers must pick up two chopsticks on the left and right at the same time ◦
 - It is stipulated that customers with odd seats should take the chopsticks on the left first, while customers with even seats should take the chopsticks on the right first ◦
- This is at the application level, solving deadlock problems based on problem characteristics; The other is to solve it at the system level

3-5 Supervisor (Monitor)

- In order to solve the problem of improper use of signals, errors are prone to occur
- The compiler is responsible for generating the code used to control access before and after generating the critical zone
- It consists of a set of initialization code, shared data objects, and functions to access those objects
- characteristics of the supervisor
 - No external function can access the area data within the supervisor
 - Procedures can enter the supervisor by calling the supervisor's function
 - Only one program can be executed within the supervisor at any one time; Processes that call other supervisor functions will be placed in a queue to wait

The syntax framework of the supervisor (Monitor)

monitor X

```
{  
    /* Shared variable declaration*/  
    int i;  
    /* conditional variable declaration*/  
    condition c;  
    /* Critical zone access function declaration*/  
    void CriticalSectionEntryPoint1();  
    void CriticalSectionEntryPoint2();  
    X  
    {  
        /* X initialization code*/  
    }  
}
```

Condition corresponding operations

- **cwait** : The process that calls the function is paused and placed in the queue for that condition variable. Waiting trips are not considered in the supervisor ◦
- **csignal** : Wake up the process waiting for the condition variable to continue execution; If there are multiple schedules paused at the same time, one of them is selected to wake up. If there is no schedule waiting, then csignal has no effect ◦

Implementation of Finite Buffers – Supervisor (1)

monitor boundedBuffer

{

 char buffer[N];

 int in, out, cnt;

 condition notempty, notfull;

void append(char x)

{

 if (cnt == n)

 notfull.cwait();

 buffer[in] = x;

 in = (in + 1) % n;

 cnt++;

 notempty.csignal();

}

Implementation of Finite Buffers – Supervisor (2)

```
void take(char *x)
{
    if (cnt == 0)
        notempty.cwait();
    *x = buffer[out];
    out = (out + 1) % n;
    cnt--;
    notfull.csignal();
}

{
    in = 0; out = 0; cnt = 0;
}
```

Implementation of Finite Buffers – Supervisor (3)

```
void producer()  
char x;  
{  
    Production of submission data x;  
    append(x);  
}
```

```
void consumer()  
char x;  
{  
    take(&x);  
    Use the retrieved data x;  
}  
}
```

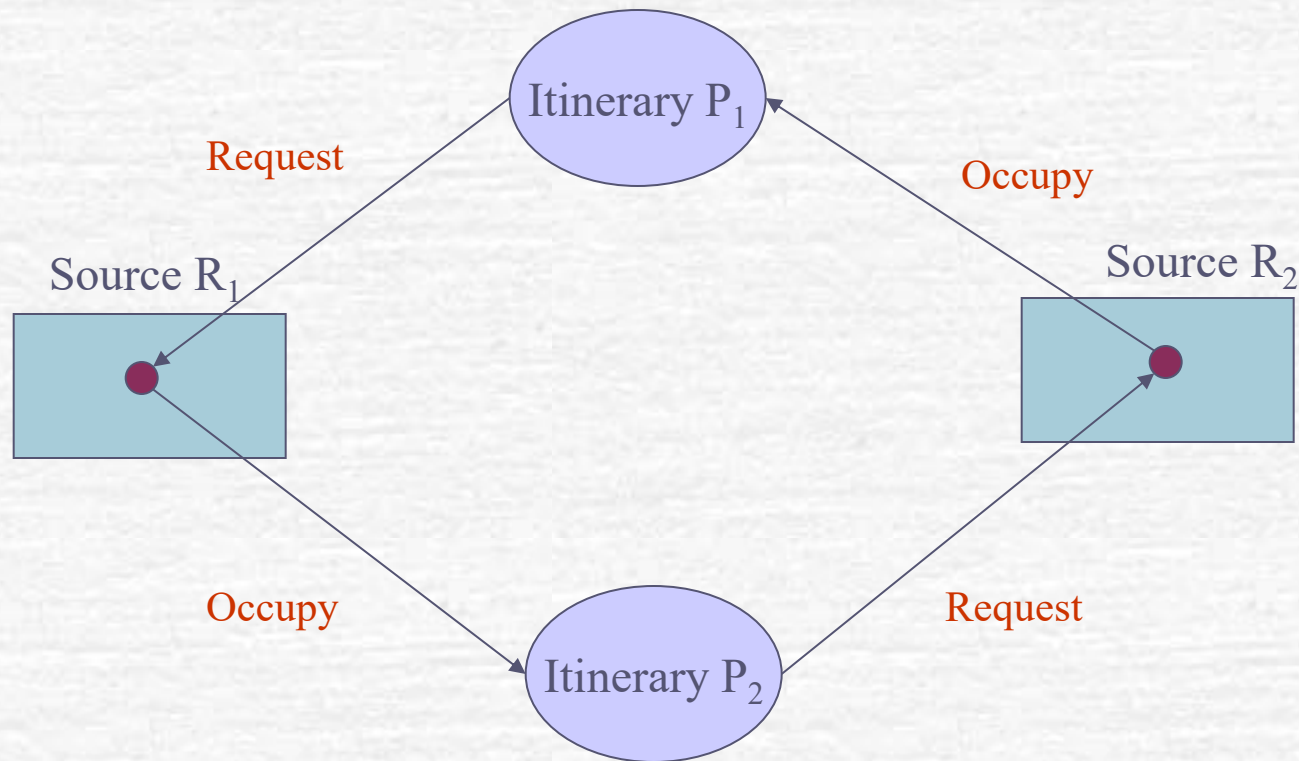
Use of supervisors

- The way to use the supervisor :
 - If you want to produce data : call `boundedBuffer.producer()`
 - If you want to consume data : call `boundedBuffer.consumer()`
- **Pros** :
 - Synchronization signals that had to be handled in the original design can now be handled by the supervisor
 - The debugging of the synchronization mechanism is limited to the scope of the supervisor

3-6 DeadLock

- As long as both parties have certain resources and need other resources in each other's hands, there may be infinite waiting, causing the problem of deadlock
- The conditions under which the deadlock occurs :
 - **Mutual exclusion** : Resources must be non-shareable
 - **Occupy and wait** : Schedules still occupy some resources while waiting for other resources
 - **Do not preemptive** : You can't force a trip that has already acquired it to take its resources
 - **Loop waiting** : Two or more strokes form a closed loop, each waiting for some resource to be occupied by the next trip in the loop

Figure 3-10 Resource allocation diagram of cyclic waiting



How to deal with deadlocks

- Prevention
- Avoid
- Detection

Prevent DeadLock

- **Prevent mutual exclusion** : Not feasible; Resources that cannot be shared at the same time must be mutually exclusive
- **Prevent occupancy and waiting** : Ensure that each process can only request resources if they do not consume them
- **Prevention cannot be preempted** : Only for resources where state can be easily saved
- **Prevent loop waiting** : Define a linear priority order for resources, and the specification process must request resources in order of priority

Avoid deadlocks

- If you determine that a provisioning request will cause a deadlock, do not proceed with the configuration
- **Secure state**: The system is able to allocate resources for each process in a certain secure execution sequence and avoid deadlocks
- **Secure execution sequence** : It must be ensured that "the number of resources required by the process at the back of the sequence must be less than the number of resources that the system does not currently have allocated, plus the total number of resources held by all processes in front of it"
 - When such a security sequence exists in the system $\langle P_1, P_2, \dots, P_n \rangle$ Time , We can be sure when the trip P_1 to P_i ($1 \leq i < n$) By the end of the trip, the trip P_{i+1} will definitely have all the resources it needs
- When such a sequence cannot be found in the system, it means that the system is in an unsafe state, that is, there is a **possibility** of deadlock

Figure 3-11 Secure resource allocation

Itinerary	The total number of R required	Number of R held	The number of R is also needed at this time
P ₁	10	5	5
P ₂	3	2	1
P ₃	8	2	6

The total number of R owned in the system = 12, and the total number of R currently idle = 3

Secure execution sequence

Itinerary	The total number of R that is idle after allocation	執行完畢後，閒置的R總數
P ₂	2	5
P ₁	0	10
P ₃	4	12

Figure 3-12 Insecure resource allocation

Itinerary	The total number of R required	Number of R held	The number of R is also needed at this time
P ₁	10	5	5
P ₂	3	2	1
P ₃	8	3	5

Total R owned in the system = 12, total number of R currently idle = 2

Possible configuration situations :

Itinerary	The total number of R that is idle after allocation	The total number of R idle after execution
P ₂	1	4

Because 4 is less than the amount of 5 required for P₁ and P₃, it is stuck in a deadlock state.

Classroom exercises

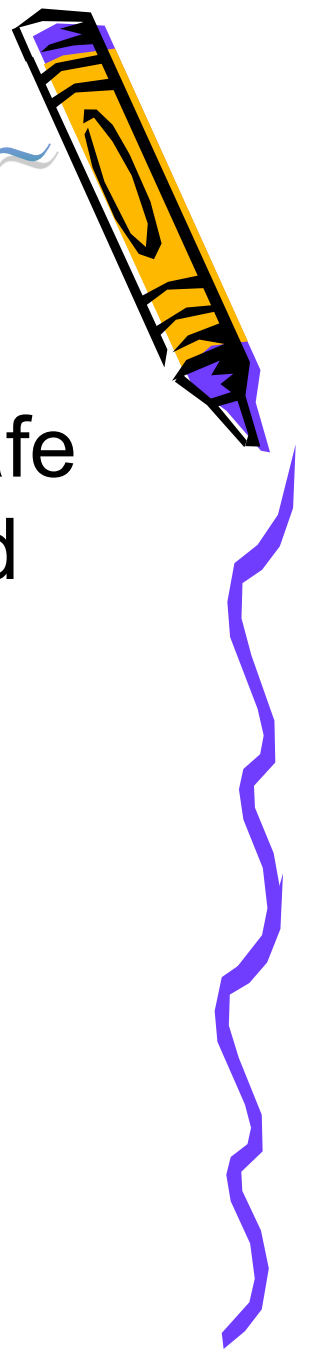
- Based on the resource requirements and configuration of the process, determine whether the following systems are in a secure state; If so, write its safe execution sequence

System 1: The total number of resource R is 11

P	Total R	Hold R	Less R
P_1	10	6	4
P_2	3	1	2
P_3	8	2	6

System 2: The total number of resource R is 11

P	Total R	Hold R	Less R
P_1	10	1	9
P_2	8	7	1
P_3	7	2	6



Practice Answers

- **System 1:** In an unsafe state, the safe execution sequence cannot be found
- **System 2:** In a safe state, the safe execution sequence is $P_2 \rightarrow P_3 \rightarrow P_1$



Banker's algorithm

- Use the concept of safe state to avoid deadlock problems by preventing the system from entering an unsafe state
- It mainly consists of two parts
 - Security algorithms
 - Resource request algorithm

The data structure of the banker's algorithm

- **Available[R]** : The number of resources currently available
- **Allocation[P][R]** : The number of resources currently allocated for each trip
- **Max[P][R]** : The maximum number of resources required per journey
- **Need[P][R]** : The number of resources that are needed after each trip ($\text{Need} = \text{Max} - \text{Allocation}$)

Security algorithms

1. Create an Array $Work[R]$ and $Finish[P]$
and set the initial value : $Work=Available$,
 $Finish[i]=false$ ($i=0$ to $P-1$)
2. Looking for i , I can be satisfied $Finish[i]=false$,
And $Need[i] \leq Work$;
If you can't find the i , skip to step 4
3. $Work = Work + Allocation[i]$;
 $Finish[i] = true$;
Skip back to step 2
4. If **all** $Finish[i]=true$, the system is in a safe state
Otherwise, it indicates an unsafe state.

Resource request algorithm

1. Create an array $\text{Request}[P][R]$
 $\text{Request}[i][j]=k$ represents the itinerary P_i , I want to request k j th resources.
2. If $\text{Request}[i] \geq \text{Need}[i]$, The end of the game
3. If $\text{Request}[i] > \text{Available}$, Then wait
4. Simulate the situation after configuration :
 $\text{Available} = \text{Available} - \text{Request}[i];$
 $\text{Allocation}[i] = \text{Allocation}[i] + \text{Request}[i];$
 $\text{Need}[i] = \text{Need}[i] - \text{Request}[i];$
5. Execute security algorithms to verify that the system is secure ◦
6. If it is not safe, then
 P_i must wait, and
will Available 、 $\text{Allocation}[i]$ 、 $\text{Need}[i]$ Revert to the value before the simulation configuration

Figure 3-13 Example of banker's algorithm

Itinerary	The total number of R required			Number of R held Allocation			The total number of R that the system has not yet configured Available		
	Max								
	R ₁	R ₂	R ₃	R ₁	R ₂	R ₃	R ₁	R ₂	R ₃
P ₁	8	0	2	1	0	0	3	1	2
P ₂	5	2	3	3	0	0			
P ₃	1	2	4	0	1	2			
P ₄	7	7	7	2	5	2			
P ₅	3	3	5	0	2	3			

Table status Security status, security sequence exists : $P_5 \rightarrow P_3 \rightarrow P_2 \rightarrow P_1 \rightarrow P_4$

(a) P5 after (3, 1, 1). **→ Security state, where a security sequence exists :**

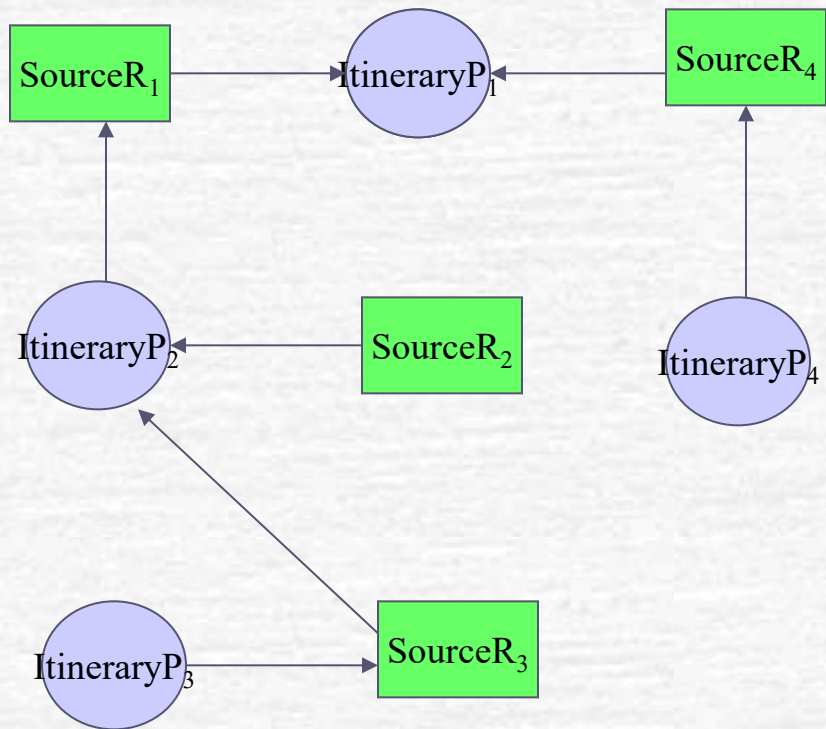
$P_5 \rightarrow P_3 \rightarrow P_2 \rightarrow P_1 \rightarrow P_4$ **→ Allow configuration**

(b) P4 after (2, 1, 2). **→ Unsafe state** , No security sequence exists **→ Configuration is not allowed**

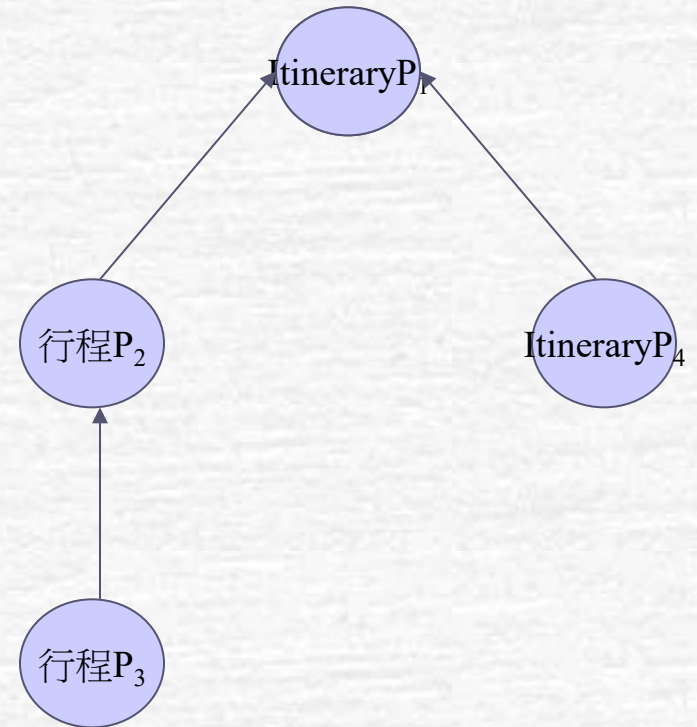
Detect deadlocks

- If the system does not prevent or avoid deadlocks, it can also provide deadlock detection and recovery to ensure the normal execution of all processes
- When there is only one copy of each resource in the system, we can use the wait graph to detect deadlocks
 - A wait graph removes the resource endpoint of a resource map, and $P_i P_j$ indicates that P_i is waiting for a resource owned by P_j
 - When a closed loop forms between certain processes in the wait graph, it means that a deadlock has formed between these processes
- If the quantity of each resource exceeds one, it must be judged using a security algorithm similar to that of a banker

Figure 3-14 Resource allocation diagram and waiting diagram



a. Resource allocation diagram



b. Waiting for the picture

Deadlock detection for multiple resources

- Similar to security algorithms for deadlock detection
- The required data structure :
 - $\text{Available}[R]$: The number of resources currently available
 - $\text{Allocation}[P][R]$: The number of resources currently allocated for each trip
 - $\text{Need}[P][R]$: The number of resources currently required for each trip
- Since no prevention is required, Max's data is not required, and $\text{Need}[P][R]$ here represents the number of resources currently required for each process

Algorithm for deadlock detection

1. Create an array **Work[R]** and **Finish[P]** ◦ Order **Work** = **Available** Pair $i=0$ to $P-1$, If **Allocation[i] $\neq 0$** , 則 **Finish[i]=false** Otherwise , **Finish[i] = true**
2. Looking for i , I can be satisfied **Finish[i]=false** , And **Need[i] $\leq$ Work** ;

If you can't find the i , skip to step 4

3. **Work = Work + Allocation[i];**
Finish[i] = true;

Skip back to step 2

4. If all **Finish[i]** , There is **Finish[i]=false**

It means that there must be a deadlock in the system, and P_i must be stuck in a deadlock state

The deadlock is lifted

- Termination of the trip
 - Want to terminate a trip or all trips?
 - What trips do you want to terminate?
- Recycled resources
- Which trips to recycle ?
 - How to go back : When a process resource is reclaimed, the process must be reverted to a previous safe state to ensure that the subsequent execution is correct
 - How to avoid trip hunger?

Ostrich algorithm

- Whether it is the prevention, avoidance, detection and recovery mechanism of deadlocks, it is quite complex and time-consuming
- In fact, we haven't found any good ways to deal with deadlocks
- For most operating systems, deadlocks are rare
- Most operating systems currently ignore the deadlock problem and pretend that the system has never had a deadlock